ECE 685D HW5

1 Problem 1: GAN (30 pts)

In this question, you will implement a GAN for generating MNIST samples.

1.1 Implement a Deep Convolutional GAN (DCGAN) (15 pts)

The architecture of the DCGAN, including both the generator and discriminator, is illustrated below:

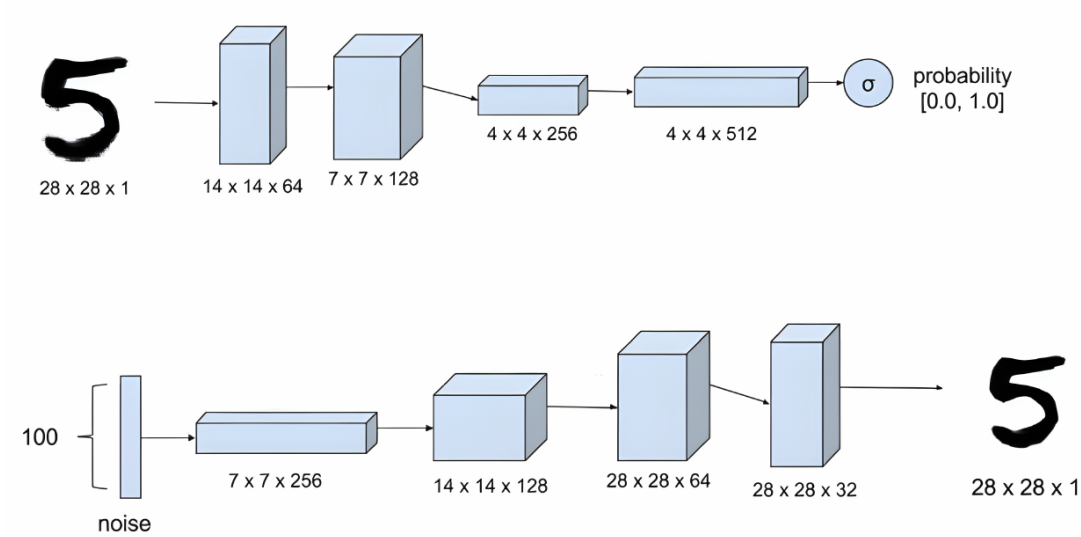

Figure 1: Model Architecture of the Discriminator and Generator

Assume the noise $z \sim N(0, I)$. Use ONLY the TRAIN SPLIT from the MNIST dataset to train your GAN with the min-max loss, defined as follows:

$$\ell = \mathbb{E}_x[\log(D(x))] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

In this equation, the first term represents is only associated with the discriminator, while the second term affect both the generator and discriminator. The generator will try to minimize the function while the discriminator tries to maximize it.

You are free to choose any activation function and may incorporate Dropout and/or Batch Normalization between layers, as these techniques can theoretically enhance convergence. Additionally, you can normalize your data to ensure pixel intensity values fall within either the range $[0, 1]$ or $[-1, 1]$.

After training the model, utilize the generator to produce 12 new samples.

1.2 Effect of Latent Space Dimension (10pts)

In this section, you will explore how the dimension of the latent variable affects the generated samples.

Train three GAN models with latent dimensions:

$$d_z \in \{16, 64, 128\}$$

For each model:

- Generate 12 samples using the trained generator
- Compare the visual quality and diversity of the generated images

Briefly describe how the latent dimension impacts:

- image quality
- diversity of samples

1.3 Interpolation in Latent Space (5pts)

Select two random latent vectors z_1 and z_2. Generate a sequence of interpolated vectors:

$$z(t) = (1-t)z_1 + tz_2, \quad t \in \{0, 0.25, 0.5, 0.75, 1\}$$

For each interpolated vector, generate the corresponding image using the generator.

Plot the generated images and describe what you observe. Does the transition between images appear smooth?

2 Problem 2: Gaussian-Bernoulli Restricted Boltzmann Machines (30 pts)

Let the energy function of Gaussian-Bernoulli RBM take the form:

$$E(v, h; \theta) = -\left(\sum_i \sum_j W_{ij} h_j \frac{v_i}{\sigma_i} - \sum_i \frac{(v_i - b_i)^2}{2\sigma_i^2} + \sum_j \alpha_j h_j \right).$$

2.1 Derivations (15pts)

Let Z be the normalization factor, using the definition of $p(v, h) = \exp(-E(v, h))/Z$, derive the mathematical expression of $p(v_i = x|h)$ and $p(h_j = 1|v)$. You may leave the expression of $p(v_i = x|h)$ in an integral form once you cancel all terms involving $\sum_j \alpha_j h_j$.

Hint: you can apply Bayes rule to get $p(v_i, |h) = \dfrac{p(v_i, h)}{p(h)}$, and then simplify your expression.

2.2 GB-RBM on KMNIST (15pts)

Train a Gaussian-Bernoulli RBM over KMNIST (with default train-test split) using CD-5 (see the lecture notes for details). Setting lr = 0.001, batch size = 128, weight decay = 0 with Adam optimizer as your hyper-parameter configuration. Then, train three versions of RBM with $M = \{16, 64, 256\}$, where M is the dimension of the hidden weights W, for 25 epochs.

On the test set for RBM with M=16, you should expect an output looking like this:

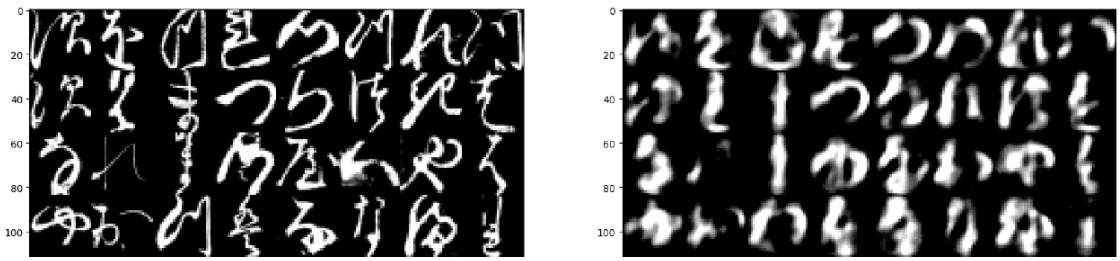

Figure 2: A sample output for M = 16 for reconstructed test set (right), you should expect visibly better results for M=256

After your model training, make a plot as in Fig. 2 for both images in the train set and test set. Then, compute and report the mean squared reconstruction error (MSE) for inferences on both datasets. A template containing the RBM has been provided to you that you can simply fill the missing lines. However, also feel free to make any modification to other parts of the template.

3 Problem 3: Two Variational Autoencoders (40 points)

Recall that for a given data point $x \in \mathbb{R}^D$, the evidence lower bound (ELBO) is given by:

$$ELBO = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}[q_\phi(z|x)||p(z)] \tag{1}$$

Here $z \in \mathbb{R}^M$, $M < D$ denotes the latent variable, $p(z)$ is the prior distribution over z, $q_\phi(z|x)$ is the variational posterior distribution (i.e., encoder output), and $p_\theta(x|z)$ is the likelihood function (i.e., decoder output). D_{KL} denotes the Kullback-Liebler divergence and $\mathbb{E}$ is the expectation.

In a standard VAE, the variational posterior is assumed to belong to a class of normal distributions, i.e., $q_\phi(z|x) = N\ (\mu_x, \sigma_x)$, whereas the prior is assumed to be standard normal, that is $p(z) = N\ (0, I)$. As a further simplification, the covariance matrix of the variational posterior can be assumed to be diagonal.

In conditional VAE, the generative model is conditioned on a data category c (or any variable we want to control). Here, we want to learn parameters (θ, ϕ) such that $p_\theta(x|c)$ generated by the generative model approximates $p(x|c)$ for each category. The decoder output, $q_\phi(z|x, c)$ approximates $p_\theta(z|x, c)$ for all x, c. You will implement a standard VAE and a C-VAE for Fashion MNIST in this problem.

3.1 Vanilla VAE (15 pts)

Train a standard VAE over the Fashion MNIST dataset. Use the standard training and testing split available in PyTorch. A template with both the model architecture and the hyperparameters has been provided to you and you should not change them if they are explicitly defined. Use 20% of the training dataset as a validation set to tune the hyperparameters of your model. Save your model weights for later use.

3.2 C-VAE (10 pts)

Train a class-conditional VAE with the same configuration as in 3.1. Also save your model weights for later use.

3.3 Manifold Comparison (15 pts)

Now you will visualize the latent representation learned by your VAE and c-VAE. You will perform the manifold visualization with the sklearn.manifold.TSNE package. Specifically, you will

1. pass the samples in the testing split through the encoder of the VAE and c-VAE
2. keep the μ obtained from VAE and c-VAE
3. use sklearn.manifold.TSNE to map the μ to 2D manifold
4. plot the 2D manifold for VAE and c-VAE and color the data points by class

Compare the data distribution on the two manifolds and described what you observed. Make a hypothesis on why you observed this phenomenon (but you don't have to prove it).